

**Asignación III informe de investigación y defensa**

Daniel Garrido

UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

Lenguaje y compiladores 2026-I

Félix Márquez

11 de julio de 2026

## **Introducción**

Los lenguajes de programación permiten que las personas puedan comunicarse con una computadora mediante un conjunto de reglas bien definidas. Para que un compilador o un intérprete pueda comprender un programa, es necesario que dicho lenguaje tenga una estructura formal que indique qué expresiones son válidas y cómo deben interpretarse.

En este contexto, los lenguajes y las gramáticas formales constituyen la base teórica sobre la cual se desarrollan los compiladores. Conceptos como alfabetos, palabras, lenguajes formales y reglas de producción permiten describir matemáticamente la estructura de un lenguaje y facilitan su procesamiento automático.

Uno de los aportes más importantes en este campo es la Jerarquía de Chomsky, la cual clasifica las gramáticas según su capacidad para describir distintos tipos de lenguajes. Esta clasificación permite comprender qué modelos son adecuados para tareas como el análisis léxico, el análisis sintáctico y la validación de estructuras complejas dentro de un programa.

Asimismo, las expresiones regulares y los autómatas finitos representan herramientas fundamentales para el reconocimiento de patrones, siendo ampliamente utilizados en compiladores, validación de datos y procesamiento de texto.

El objetivo de este informe es estudiar los principales conceptos relacionados con los lenguajes y gramáticas formales, analizar la Jerarquía de Chomsky y aplicar estos conocimientos mediante ejemplos prácticos de derivaciones, optimización de gramáticas y construcción de expresiones regulares y autómatas finitos.

## Relación entre Gramáticas y Lenguajes Formales

Una **gramática formal** es un conjunto de reglas que define cómo pueden construirse las cadenas válidas de un lenguaje. Estas reglas indican la estructura que debe seguir una secuencia de símbolos para ser considerada correcta.

Por otra parte, un **lenguaje formal** es el conjunto de todas las cadenas que pueden generarse aplicando las reglas definidas por una gramática. En otras palabras, la gramática establece las reglas de construcción y el lenguaje está formado por todos los resultados que cumplen dichas reglas.

Esta relación es similar a una receta de cocina: la receta indica los pasos para preparar un plato, mientras que el plato terminado es el resultado de seguir esas instrucciones. De igual forma, una gramática no contiene todas las cadenas posibles, sino las reglas necesarias para construirlas.

En los lenguajes de programación este concepto es fundamental, ya que un compilador utiliza la gramática del lenguaje para verificar si el código fuente fue escrito correctamente. Si una instrucción puede generarse siguiendo las reglas de la gramática, el compilador la considera válida; en caso contrario, reportará un error de sintaxis.

### Ejemplo práctico

Supongamos un lenguaje de programación muy sencillo en el que únicamente se permiten declaraciones de variables de tipo **int** o **float**.

La gramática puede definirse de la siguiente manera:

#### Símbolo inicial:

S

#### Producciones:

- $S \rightarrow \text{TIPO ID ;}$

- $\text{TIPO} \rightarrow \text{int} \mid \text{float}$
- $\text{ID} \rightarrow \text{edad} \mid \text{salario} \mid \text{contador}$

A partir de estas reglas pueden generarse las siguientes instrucciones:

- `int edad;`
- `int contador;`
- `float salario;`

Estas instrucciones forman parte del lenguaje porque cumplen con la estructura definida por la gramática.

¿Cómo la gramática genera una instrucción?

Supongamos que queremos obtener la instrucción:

**`int edad;`**

El proceso de derivación sería el siguiente:

1. Se inicia con el símbolo inicial **S**.
2. Se aplica la producción  $\text{S} \rightarrow \text{TIPO ID ;}$ , obteniendo:  
**TIPO ID ;**
3. Se reemplaza **TIPO** utilizando la producción  $\text{TIPO} \rightarrow \text{int}$ :  
**int ID ;**
4. Finalmente se reemplaza **ID** utilizando la producción  $\text{ID} \rightarrow \text{edad}$ :  
**int edad;**

En este punto ya no existen símbolos por reemplazar, por lo que la derivación ha finalizado y se obtiene una instrucción válida del lenguaje.

Este mismo procedimiento permite generar otras instrucciones, como **int contador;** o **float salario;**, demostrando que la gramática define las reglas y el lenguaje corresponde al conjunto de todas las cadenas que pueden construirse a partir de ellas.

### **Jerarquía de Chomsky**

Como se explicó anteriormente, una gramática define las reglas para construir un lenguaje formal. Sin embargo, no todos los lenguajes tienen el mismo nivel de complejidad. Algunos pueden describirse mediante reglas muy simples, mientras que otros requieren reglas mucho más elaboradas para representar estructuras complejas, como bloques de código, expresiones matemáticas o programas completos.

Con el objetivo de clasificar estas diferencias, el lingüista **Noam Chomsky** propuso la **Jerarquía de Chomsky**, una clasificación que organiza las gramáticas según su poder de expresión. Esta jerarquía está compuesta por cuatro tipos de gramáticas, donde cada nivel puede describir lenguajes más complejos que el anterior, aunque también requiere mecanismos de procesamiento más sofisticados.

En el desarrollo de compiladores esta clasificación tiene una gran importancia. Por ejemplo, las gramáticas más simples se utilizan durante el análisis léxico para reconocer identificadores, números o palabras reservadas, mientras que las gramáticas libres de contexto permiten analizar la estructura completa de un programa, verificando que instrucciones como **if**, **while** o bloques delimitados por llaves estén correctamente escritos.

**Tabla 1**

*Tipos de gramáticas de la Jerarquía de Chomsky*

| Tipo   | Nombre                           | Características                                                                 | Uso principal                              |
|--------|----------------------------------|---------------------------------------------------------------------------------|--------------------------------------------|
| Tipo 0 | Gramáticas Irrestringidas        | No poseen restricciones sobre las reglas de producción. Son las más expresivas. | Modelos computacionales generales.         |
| Tipo 1 | Gramáticas Sensibles al Contexto | Las reglas de producción dependen del contexto donde aparece un símbolo.        | Lenguajes con restricciones complejas.     |
| Tipo 2 | Gramáticas Libres de Contexto    | Cada producción reemplaza un único símbolo no terminal.                         | Sintaxis de los lenguajes de programación. |
| Tipo 3 | Gramáticas Regulares             | Poseen reglas muy simples y fáciles de procesar.                                | Análisis léxico y expresiones regulares.   |

### Notación BNF (Backus-Naur Form)

Esta notación permite describir las producciones de una gramática mediante reglas claras y legibles, siendo ampliamente utilizada en la definición de lenguajes de programación.

En BNF, una gramática está formada por **símbolos no terminales** y **símbolos terminales**:

- Los **símbolos no terminales** representan elementos que todavía pueden ser desarrollados o reemplazados por otras reglas de la gramática. Normalmente se escriben entre símbolos **< >**.
- Los **símbolos terminales** representan los elementos finales que aparecerán directamente en una cadena válida del lenguaje, como palabras reservadas, operadores o caracteres específicos.

Una regla de producción en BNF tiene la siguiente estructura:

**<simbolo\_no\_terminal> ::= expresion**

El operador **::=** significa "**se define como**" o "**puede ser reemplazado por**". Por lo tanto, indica las posibles formas en que un símbolo no terminal puede transformarse.

## Tipo 0 - Gramática Irrestricta

Estas gramáticas no poseen restricciones importantes sobre sus reglas de producción y permiten definir prácticamente cualquier lenguaje que pueda ser procesado por una máquina de Turing. Debido a su gran capacidad expresiva y complejidad, no suelen utilizarse directamente para definir lenguajes de programación.

Ejemplo:

$$\langle S \rangle ::= a \langle S \rangle b \mid ab$$

Esta regla permite generar cadenas donde la cantidad de símbolos **a** y **b** mantiene una relación determinada.

## Tipo 1 - Gramática Sensible al Contexto

En este tipo de gramáticas, la transformación de un símbolo depende del contexto en el que aparece. Es decir, una regla puede considerar los símbolos que rodean al elemento que será reemplazado.

Ejemplo:

$$\langle aB \rangle ::= \langle ab \rangle$$
$$\langle Bc \rangle ::= \langle bc \rangle$$

Estas reglas representan transformaciones donde el símbolo **<B>** no puede analizarse de forma independiente, sino considerando los elementos que aparecen alrededor.

Este tipo de gramática se utiliza para representar lenguajes donde existen restricciones que dependen del contexto.

## Tipo 2 - Gramática Libre de Contexto

Las gramáticas libres de contexto son las más utilizadas en el diseño de compiladores, ya que permiten representar la estructura sintáctica de los lenguajes de programación.

En este tipo de gramáticas, cada regla reemplaza un único símbolo no terminal sin considerar los elementos que lo rodean.

Ejemplo:

$\langle S \rangle ::= \langle TIPO \rangle \langle ID \rangle ;$

$\langle TIPO \rangle ::= \text{int} \mid \text{float}$

$\langle ID \rangle ::= \text{edad} \mid \text{salario} \mid \text{contador}$

En este ejemplo:

- $\langle S \rangle$ ,  $\langle TIPO \rangle$  e  $\langle ID \rangle$  son símbolos no terminales, porque representan estructuras que todavía deben ser desarrolladas.
- $\text{int}$ ,  $\text{float}$ ,  $\text{edad}$ ,  $\text{salario}$ ,  $\text{contador}$  y  $;$  son símbolos terminales, porque aparecen directamente en la instrucción final.

Aplicando estas reglas pueden generarse instrucciones válidas como:

$\text{int edad};$

$\text{int contador};$

$\text{float salario};$

Este tipo de gramática permite describir declaraciones de variables, expresiones, estructuras de control y otros elementos sintácticos presentes en los lenguajes de programación.



### Tipo 3 - Gramática Regular

Las gramáticas regulares son las más simples dentro de la Jerarquía de Chomsky. Sus reglas poseen una estructura limitada, lo que permite analizarlas de manera eficiente mediante autómatas finitos.

Son utilizadas principalmente durante el análisis léxico de un compilador para reconocer patrones como identificadores, números, operadores o palabras reservadas.

Ejemplo:

$$\langle S \rangle ::= a \langle A \rangle$$
$$\langle A \rangle ::= b \langle B \rangle$$
$$\langle B \rangle ::= c$$

Esta gramática genera la cadena:

abc

Demostrando cómo una secuencia de reglas simples puede producir una cadena válida dentro del lenguaje definido.

### Derivación y Modelado (Caso Práctico)

Con el objetivo de aplicar los conceptos estudiados sobre Gramáticas Libres de Contexto (GLC), se diseñó un pequeño lenguaje de dibujo utilizando el alfabeto  $\Sigma = \{a, c, g, t\}$ . En este lenguaje, cada símbolo representa una instrucción gráfica sencilla que posteriormente será interpretada por un programa desarrollado en Python. De esta manera, es posible observar cómo una gramática no solo permite definir un lenguaje, sino también generar secuencias que pueden transformarse en acciones concretas.

Para este caso práctico se asignó el siguiente significado a cada símbolo del alfabeto:

**Tabla 2**

*lenguaje de dibujo*

| Símbolo | Acción                                     |
|---------|--------------------------------------------|
| a       | Avanzar                                    |
| c       | Girar 90°                                  |
| g       | Desplazarse para comenzar una nueva figura |
| t       | Finalizar el dibujo                        |

*A partir de estas instrucciones se definió la siguiente Gramática Libre de Contexto:*

<INICIO> ::= <CUADRADO>

| <TRIANGULO>

| <LINEA>

| <ARBOL>

| <CUBO>

<CUADRADO> ::= acacacact

<TRIANGULO> ::= acacact

<LINEA> ::= aaaat

<ARBOL> ::= agaagat

<CUBO> ::= acacacactgacacacact

La gramática anterior establece que el símbolo inicial **<INICIO>** puede derivar en cualquiera de las cinco figuras propuestas. Cada figura está representada por una secuencia de símbolos que posteriormente será interpretada por el programa como un conjunto de instrucciones de dibujo.

A continuación se presentan las derivaciones correspondientes a cada una de las figuras.

#### **Derivación 1: Cuadrado**

<INICIO> ↓ <CUADRADO> ↓ acacacact

#### **Derivación 2: Triángulo**

<INICIO> ↓ <TRIANGULO> ↓ acacact

#### **Derivación 3: Línea**

<INICIO> ↓ <LINEA> ↓ aaaat

#### **Derivación 4: Árbol**

<INICIO> ↓ <ARBOL> ↓ agaagat

#### **Derivación 5: Cubo**

<INICIO> ↓ <CUBO> ↓ acacacactgacacacact

Las cadenas obtenidas mediante estas derivaciones constituyen instrucciones válidas dentro del lenguaje diseñado. En la implementación práctica, cada símbolo será interpretado por un programa que ejecutará la acción correspondiente, permitiendo generar gráficamente las figuras propuestas y demostrando la aplicación de una Gramática Libre de Contexto en un caso sencillo de modelado.

### **Higiene y Optimización de Gramáticas**

Al diseñar una gramática no basta con que sea capaz de generar un lenguaje. También es importante que sus reglas permitan un análisis sintáctico claro y eficiente. Una gramática mal diseñada puede provocar ambigüedades, ciclos infinitos durante el análisis o dificultades para que un compilador determine qué regla debe aplicar en cada momento.

Por esta razón, durante el diseño de compiladores es común aplicar técnicas de optimización que permitan eliminar estos problemas antes de implementar el analizador sintáctico. A continuación, se presentan tres de las patologías más comunes y la forma de corregirlas.

### Gramática Ambigua

Una gramática es **ambigua** cuando una misma cadena puede derivarse de más de una forma, produciendo diferentes árboles sintácticos. Esto representa un problema para el compilador, ya que una misma expresión podría interpretarse con distintos significados.

Considérese la siguiente gramática:

$$\langle E \rangle ::= \langle E \rangle + \langle E \rangle$$
$$| \langle E \rangle * \langle E \rangle$$
$$| \text{id}$$

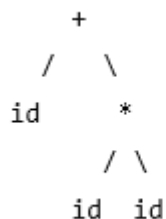
Y la siguiente expresión:

$$\text{id} + \text{id} * \text{id}$$

Esta cadena puede interpretarse de dos maneras diferentes.

### Figura 1

*Árbol de derivación 1*

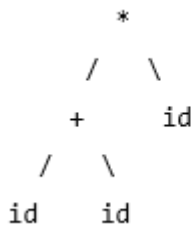


Interpretación:

$$\text{id} + (\text{id} * \text{id})$$

## Figura 2

Árbol de derivación 2



Interpretación:

$(id + id) * id$

Como la misma cadena produce dos árboles sintácticos distintos, la gramática es ambigua.

## Recursividad por la Izquierda

La recursividad por la izquierda ocurre cuando una producción comienza llamándose a sí misma. Este tipo de gramáticas puede provocar problemas en analizadores sintácticos descendentes, ya que pueden entrar en un ciclo infinito.

Ejemplo:

$$\langle E \rangle ::= \langle E \rangle + \langle T \rangle$$
$$| \langle T \rangle$$
$$\langle T \rangle ::= id$$

## Eliminación de la recursividad

Paso 1: Se identifica la producción recursiva.

$$\langle E \rangle ::= \langle E \rangle + \langle T \rangle$$

Paso 2: Se crea un nuevo símbolo auxiliar.

Gramática transformada:

$$\langle E \rangle ::= \langle T \rangle \langle E' \rangle$$
$$\langle E' \rangle ::= + \langle T \rangle \langle E' \rangle$$
$$| \epsilon$$
$$\langle T \rangle ::= \text{id}$$

Con esta transformación la gramática conserva el mismo lenguaje, pero ahora puede ser procesada correctamente por un parser descendente.

### **Factorización por la Izquierda**

La factorización por la izquierda se utiliza cuando dos o más producciones comienzan con la misma secuencia de símbolos. En estos casos, el compilador no puede decidir inmediatamente qué producción utilizar.

Considérese la siguiente gramática:

$$\langle S \rangle ::= \text{if } \langle E \rangle \text{ then } \langle S \rangle$$
$$| \text{if } \langle E \rangle \text{ then } \langle S \rangle \text{ else } \langle S \rangle$$

Ambas producciones comienzan exactamente igual:

$$\text{if } \langle E \rangle \text{ then } \langle S \rangle$$

Para evitar esta ambigüedad se extrae el prefijo común.

Gramática factorizada:

$\langle S \rangle ::= \text{if } \langle E \rangle \text{ then } \langle S \rangle \langle X \rangle$

$\langle X \rangle ::= \text{else } \langle S \rangle$

$|\varepsilon$

Con esta modificación el analizador sintáctico primero reconoce la parte común y posteriormente decide si existe o no la cláusula **else**, simplificando el proceso de análisis.

### De la Expresión Regular al Autómata (Caso Práctico: Ajedrez)

El lenguaje **PGN (Portable Game Notation)** es el formato estándar utilizado para registrar partidas de ajedrez. Cada movimiento se representa mediante una notación específica que permite almacenar y compartir partidas de manera estructurada.

Para este ejercicio se definió un subconjunto simplificado del lenguaje PGN, considerando únicamente algunos movimientos básicos con el propósito de demostrar cómo una expresión regular puede utilizarse para reconocer cadenas válidas y cómo dicha expresión puede representarse mediante un Autómata Finito Determinístico (AFD).

#### Subconjunto del lenguaje PGN

Se considerarán válidos los siguientes tipos de movimientos:

- Movimiento de un peón (ejemplo: **e4**)
- Movimiento de un caballo (ejemplo: **Nf3**)
- Movimiento de un alfil (ejemplo: **Bb5**)
- Enroque corto (**O-O**)

Ejemplos de cadenas válidas:

e4

d5

Nf3

Bb5

O-O

### Expresión Regular

La siguiente expresión regular permite reconocer el subconjunto definido anteriormente:

$$([a-h][1-8])|(N[a-h][1-8])|(B[a-h][1-8])|(O-O)$$

Esta expresión valida:

- Un movimiento de peón (casilla).
- Un movimiento de caballo.
- Un movimiento de alfil.
- El enroque corto.

### Autómata Finito Determinístico (AFD)

El funcionamiento del autómata puede resumirse de la siguiente manera:

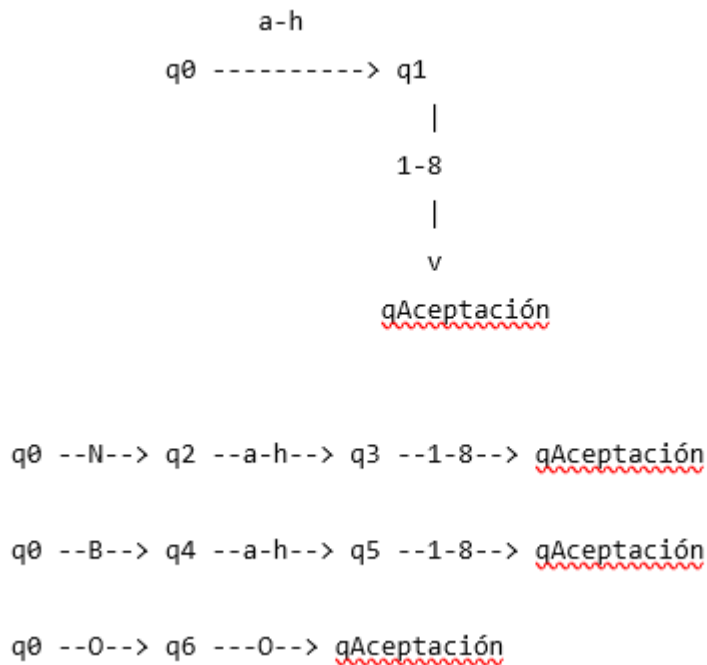
1. El autómata inicia en el estado inicial **q0**.
2. Si recibe una letra entre **a** y **h**, espera un número entre **1** y **8** para aceptar un movimiento de peón.
3. Si recibe la letra **N**, espera una columna y posteriormente una fila para aceptar un movimiento de caballo.
4. Si recibe la letra **B**, sigue el mismo procedimiento para reconocer un movimiento de alfil.
5. Si recibe la secuencia **O-O**, acepta un enroque corto.
6. Cualquier otra secuencia conduce a un estado de rechazo.



El siguiente diagrama representa de forma simplificada el comportamiento del autómata:

**Figura 3**

*comportamiento del autómata*



Este autómata permite verificar si una cadena pertenece al subconjunto del lenguaje PGN definido para este ejercicio. Si la secuencia cumple con alguno de los patrones establecidos, será aceptada; de lo contrario, será rechazada.

### Conclusiones

El desarrollo de esta investigación permitió comprender la importancia que tienen las gramáticas formales dentro del diseño e implementación de los lenguajes de programación. Se evidenció que una gramática no solo define la estructura de un lenguaje, sino que también proporciona las reglas necesarias para que un compilador pueda analizar y validar el código escrito por un programador.

Asimismo, el estudio de la Jerarquía de Chomsky permitió identificar que existen distintos tipos de gramáticas según su capacidad para describir lenguajes, comprendiendo por qué las

gramáticas regulares se utilizan en el análisis léxico, mientras que las gramáticas libres de contexto son fundamentales para describir la sintaxis de los lenguajes de programación.

Durante el desarrollo de los casos prácticos se aplicaron estos conceptos mediante el diseño de una Gramática Libre de Contexto y la construcción de expresiones regulares y autómatas finitos. Estas actividades permitieron observar cómo la teoría puede transformarse en soluciones prácticas para el reconocimiento y procesamiento de lenguajes.

Finalmente, se concluye que los fundamentos estudiados constituyen la base para el desarrollo de compiladores e intérpretes, además de tener aplicaciones en áreas como la validación de datos, el procesamiento de texto y el reconocimiento de patrones, demostrando la vigencia e importancia de estos conceptos dentro de la informática actual.

## **Referencias**

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson.

Fowler, M. (2010). *Domain-specific languages*. Addison-Wesley Professional.

Python Software Foundation. (2025). Python documentation. <https://docs.python.org/3>

The Rust Project Developers. (2025). The Rust programming language. <https://doc.rust-lang.org/book/>

The Zig Software Foundation. (2025). Zig documentation. <https://ziglang.org/documentation/>